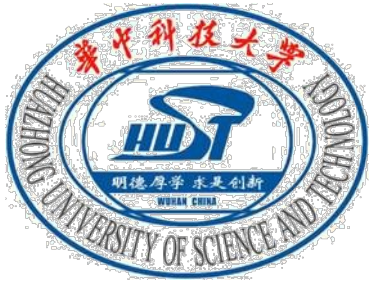
DS1302 RTC 软件设计



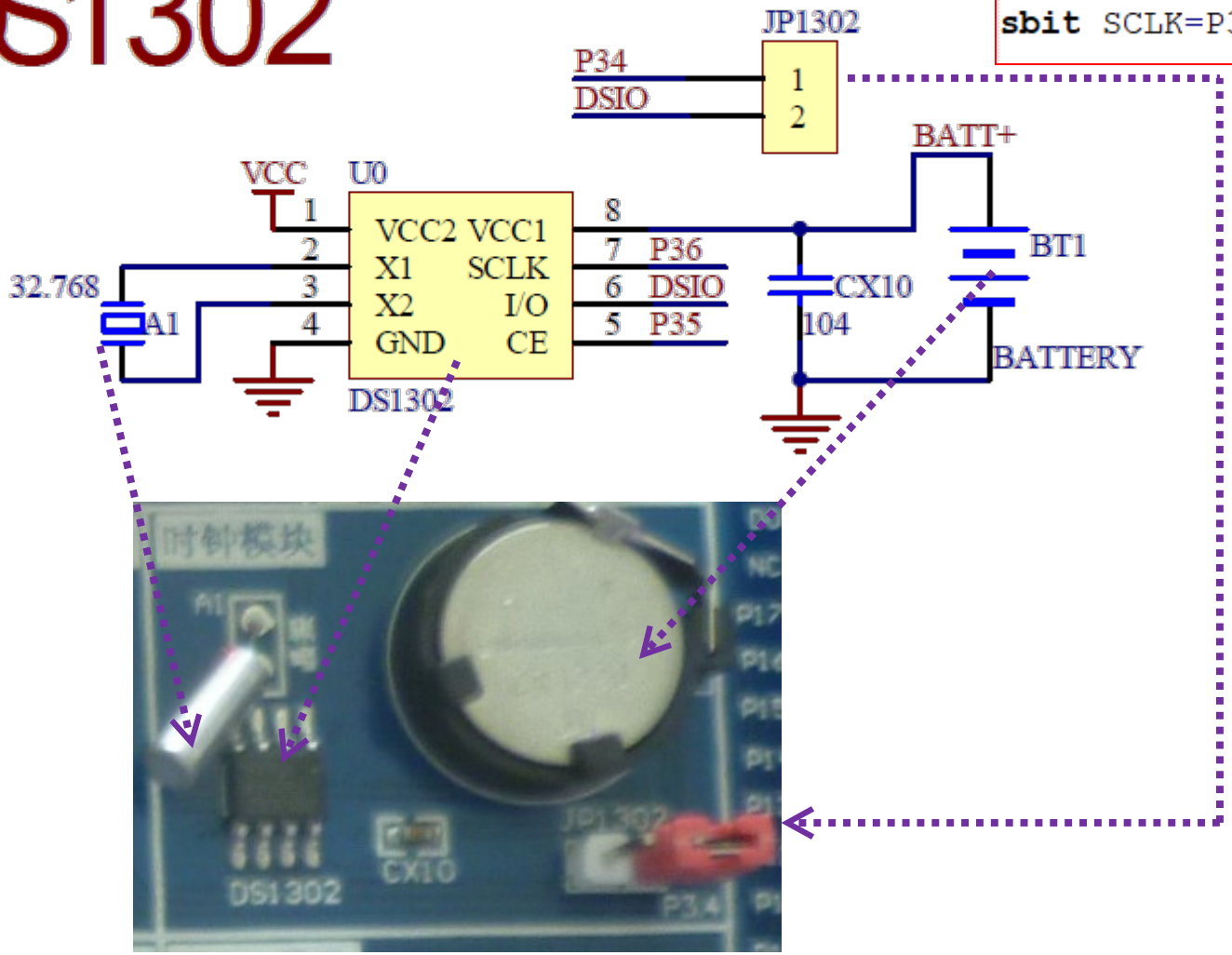
实时时钟控制器

- RTC， Real Time Controller
- RTC 基于以标准时钟信号为基准，通过对标准时钟脉冲计数，产生年月日时分秒等时间信息
- RTC 常采用纽扣电池供电，系统电源关闭时继续工作
- Datasheet 见 QQ 文件夹

51 实验板中的 RTC 电路

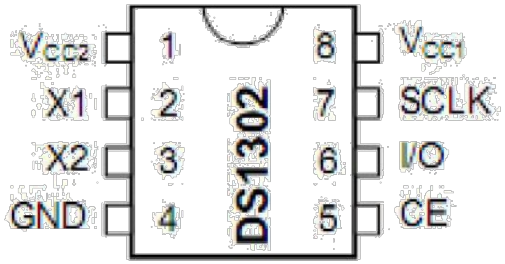
```
//---定义ds1302使用的IO口---//
sbit DSIO=P3^4;
sbit RST=P3^5;
sbit SCLK=P3^6;
```

DS1302

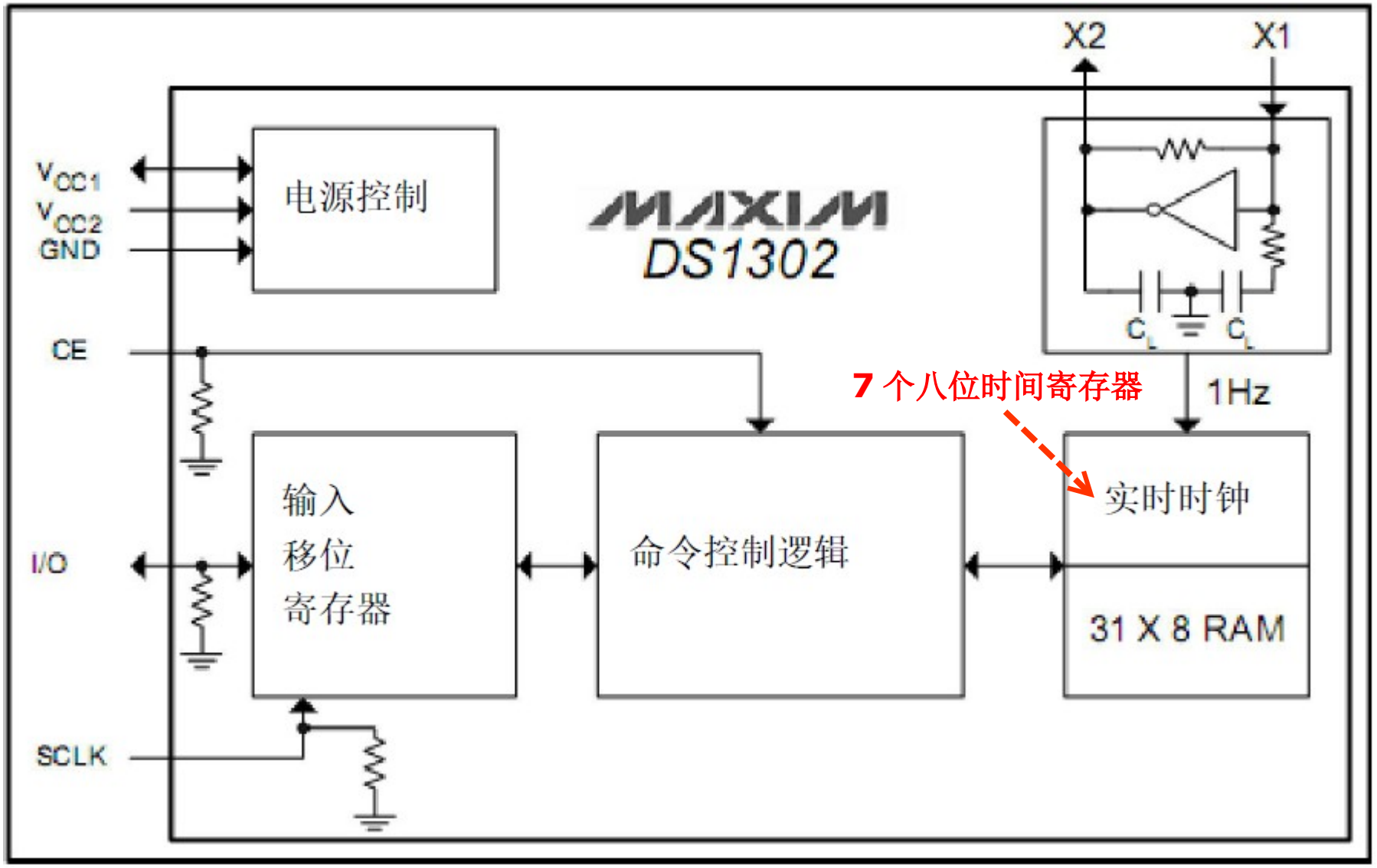


DS1302 实时时钟控制器

- 计算年、月、日、时、分、秒、星期，直到 2100 年，并有闰年调节功能
- 对于少于 31 天的月份月末会自动调整
- 有 AM/PM 标识，可工作在 12 小时制或 24 小时制
- 31 x 8 位通用暂存 RAM
- 3 线串行输入输出使管脚数最少
- 32.768KHz 时钟
- 2.0V 至 5.5V 宽电压范围操作
- 在 2.0V 时工作电流小于 300nA



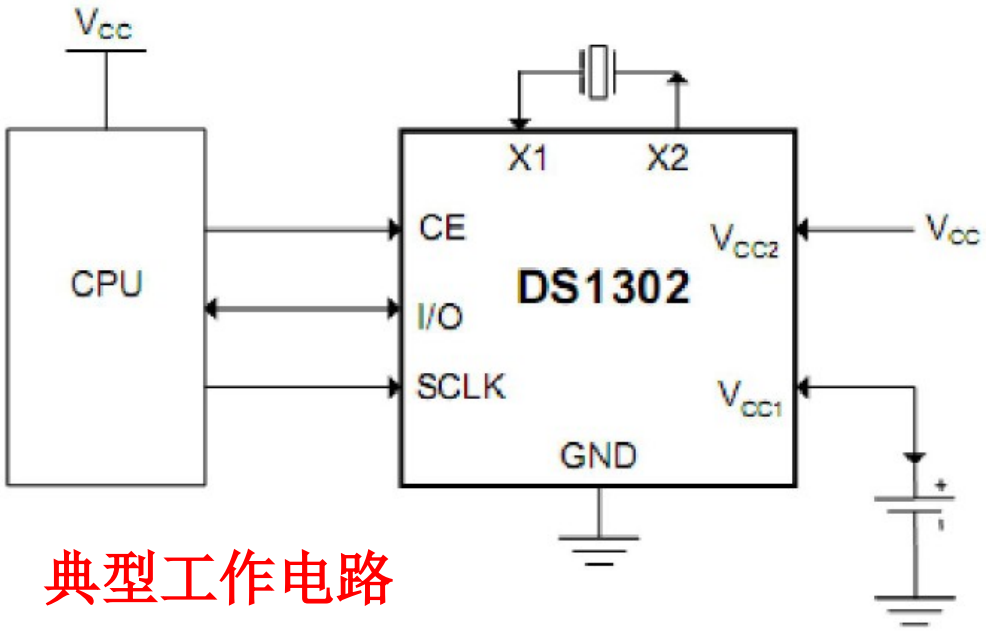
DS1302 内部结构


X2
X1
VCC1
VCC2
GND
电源控制
DS1302
CE
I/O
SCLK
输入
移位
寄存器
命令控制逻辑
实时时钟
31 X 8 RAM
1Hz
7 个八位时间寄存器

DS1302 实时时钟控制器

1	V_{CC2}	双供电配置中的主电源供应管脚，V_{CC1} 连接到备用电源，在主电源失效时保持时间和日期数据. DS1302 工作于 V_{CC1} 和 V_{CC2} 中较大者. 当 V_{CC2} 比 V_{CC1} 高 0.2V 时，V_{CC2} 给 DS1302 供电. 当 V_{CC1} 比 V_{CC2} 高时，V_{CC1} 给 DS1302 供电.
2	X1	与标准的 32.768kHz 石英晶体相连. 内部振荡器被设计与指定的 6pF 装载电容的晶体一起工作. 更多关于晶体选择和布局注意事项的信息请参考应用笔记 58 页:Dallas 实时时钟晶振注意事项. DS1302 也可以被外部的 32.768kHz 振荡器驱动. 这种配置下，X1 与外部震荡信号连接，X2 悬浮.
3	X2	
4	GND	电源地
5	CE	输入. CE 信号在读写时必须保持高电平. 此管脚内部有一个 40kΩ (典型值) 的下拉电阻连接到地. 注意:先前的数据手册修正把 CE 当作 RST. 管脚的功能没有改变.
6	I/O	输入/推挽输出. I/O 管脚是三线接口的双向数据管脚. 此管脚内部有一个 40kΩ (典型值) 的下拉电阻连接到地.
7	SCLK	输入. SCLK 用来同步串行接口上的数据动作. 此管脚内部有一个 40kΩ (典型值) 的下拉电阻连接到地.
8	V_{CC1}	低功率工作在单电源和电池工作系统和低功率备用电池. 在使用涓流充电的系统中，这个管脚连接到可再充能量源. UL 认证在使用锂电池时确保避免反向充电电流. 详见

DS1302 实时时钟控制器



典型工作电路

DS1302 命令字

- 每次数据传输都需要用一个命令字来启动
- Bit 7 必须为 1，如果为 0，禁止对 DS1302 的写操作
- Bit 6 为 0 时访问时钟 / 日历数据，为 1 时访问 RAM 数据
- Bit 1 至 Bit 5 寻址输入输出寄存器
- Bit 0 为 0 时，是写操作，为 1 时是读操作
- 命令字节 Bit 0 总为 0

7	6	5	4	3	2	1	0
1	RAM / $\overline{CK}$	A4	A3	A2	A1	A0	RD / $\overline{W}$

DS1302 寄存器

12/24 模式设置

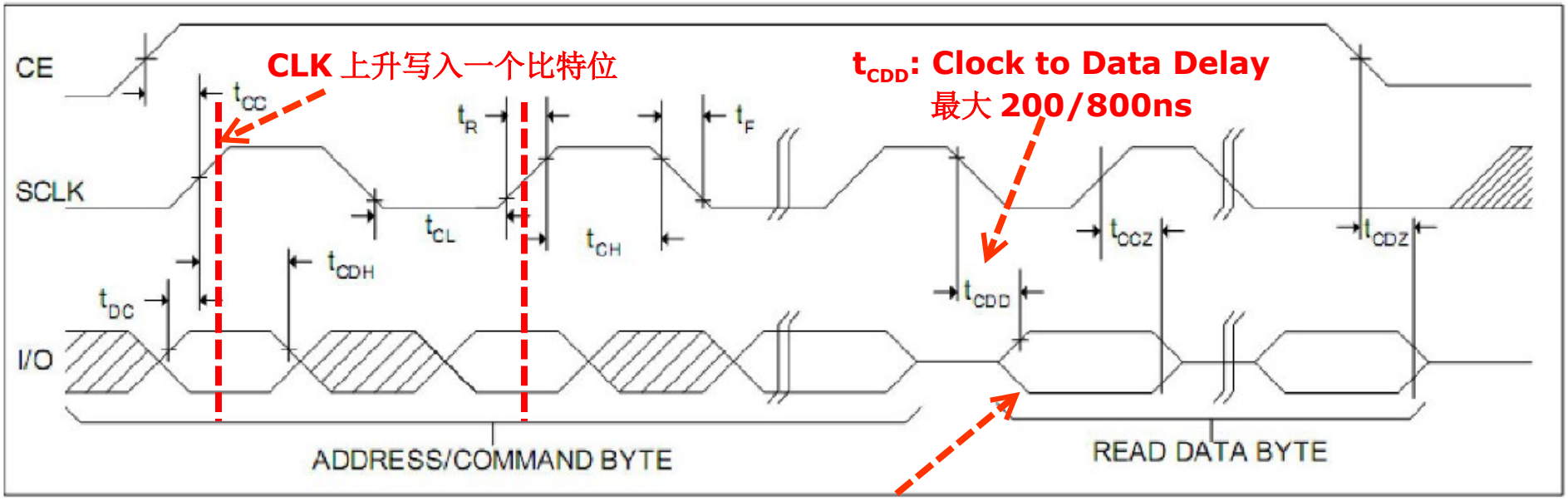
Clock Halt
时钟暂停控制位

READ	WRITE	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0	RANGE
81h	80h	CH	秒十位 10 Seconds			秒个位 Seconds				00–59
83h	82h		10 Minutes			Minutes				00–59
85h	84h	12/$\overline{24}$	0	10 $\overline{\text{AM}}$/PM	Hour	Hour				1–12/0–23
87h	86h	0	0	10 Date		Date				1–31
89h	88h	0	0	0	10 Month	Month				1–12
8Bh	8Ah	0	0	0	0	0	Day			1–7
8Dh	8Ch	10 Year				Year				00–99
8Fh	8Eh	WP	0	0	0	0	0	0	0	—
91h	90h	TCS	TCS	TCS	TCS	DS	DS	RS	RS	—

写保护

```
//---DS1302时钟初始化2013年1月1日星期二12点00分00秒。---//
//---存储顺序是秒分时日月周年,存储格式是用BCD码---//
uchar TIME[7] = {0, 0, 0x12, 0x01, 0x01, 0x02, 0x13};
                秒  分   小时    日    月    星期    年
```

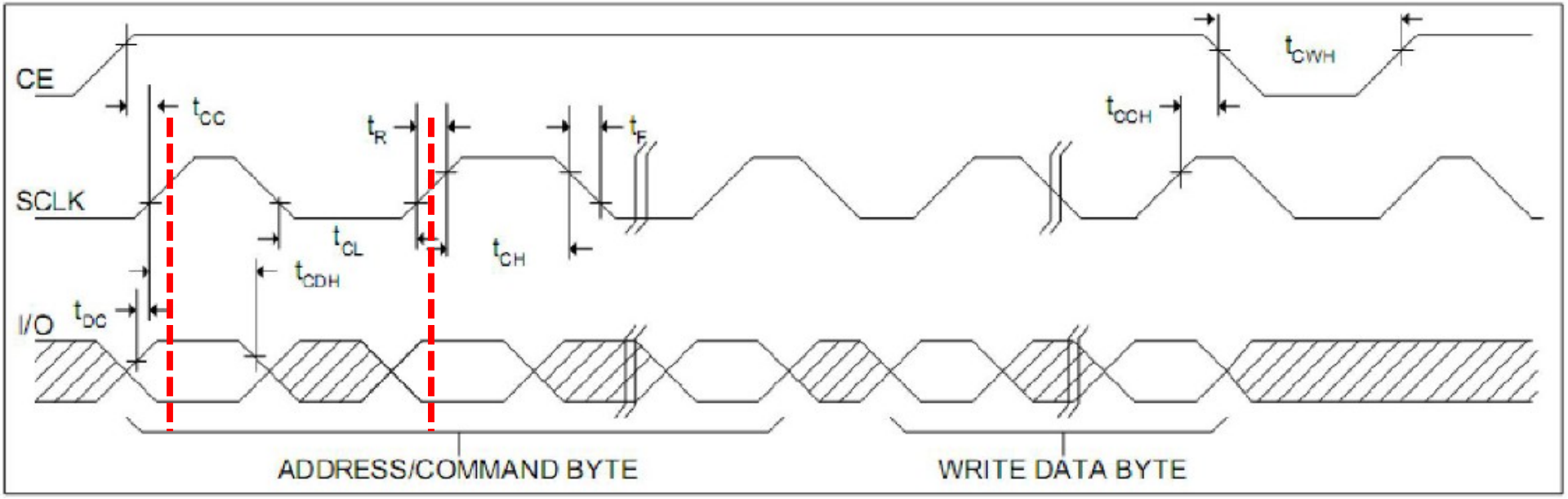
DS1302 读时序




```
//---定义ds1302使用的IO口---//
sbit DSIO=P3^4;
sbit RST=P3^5;
sbit SCLK=P3^6;
```

```
for(n=0; n<8; n++)//读取8位数据
{
    dat1 = DSIO;//从最低位开始接收
    dat = (dat>>1) | (dat1<<7);
    SCLK = 1;
    _nop_();
    SCLK = 0;//DS1302下降沿时，放置数据
    _nop_();
}
```

DS1302 写时序




```
//---定义ds1302使用的IO口---//
sbit DSIO=P3^4;
sbit RST=P3^5;
sbit SCLK=P3^6;
```

```
for (n=0; n<8; n++)//写入8位数据
{
    DSIO = dat & 0x01;
    dat >>= 1;
    SCLK = 1;//数据在上升沿时，DS1302读取数据
    _nop_();
    SCLK = 0;
    _nop_();
}
```

51 单片机 C 语音编程

- 单片机的 C 语言采用 C51 编译器 (简称 C51)
- 由 C51 产生的目标代码短、运行速度高、所需存储空间小、符合 C 语言的 ANSI 标准，生成的代码遵循 Intel 目标文件格式
- 可与 A51 汇编语言混和使用

51 单片机 C 语音编程优点

- C51 管理内部寄存器和存贮器的分配，编程时，无需考虑不同存储器的寻址和数据类型等细节问题
- 程序由若干函数组成，具有良好的模块化结构
- 有丰富的子程序库可直接引用，从而大大减少用户编程的工作量
- C 语言和汇编语言可以交叉使用
- 汇编语言程序代码短、运行速度快、但复杂运算编程耗时
- 用汇编语言编写与硬件有关的部分程序，用 C 语言编写与硬件无关的运算部分程序，可充分发挥两种语言的长处，可以提高开发效率。

C51 程序结构

- C51 的程序由一个个函数组成，函数和其他语言的“子程序”或“过程 ”具有相同的意义
- 必须有一个主函数 main()，程序的执行从 main() 函数开始，调用其他函数后返回主函数 main()，最后在主函数中结束整个程序而不管函数的排列顺序如何

```
//循环显示，检查是否有按键，如果有，对段码缓存移位，并更新20存储单元的段码
        ORG 0000H
        AJMP MAIN     //上电后执行的第一条指令
        ORG  0030H
MAIN:   MOV R0,#00H   //R0,R1用于DISPLAY函数内部循环控制,R1指向段码表首地址
        MOV R1,#20H
        ACALL DISPLAY   //显示8个数码管，此时段码为上电初始值
        MOV R4, #0FFH   //当前键码,FF是不可能出现的键码，表示没有键按下
        ACALL KEYDOWN
        CJNE R4, #0FFH, CONTINUE //键码不等于FF，有键被按下
        AJMP MAIN
CONTINUE:  ACALL SHIFT   //调用子程序，把八个段码移一位
        MOV A, R4     //R4是键码
        MOV DPTR,#DIG_CODE
        MOVC A, @A+DPTR
        MOV 20H,A       //把当前按键键码对应段码写入20H存储单元
        AJMP MAIN
```

```
void main()
{
    Ds1302Init();
    while(1)
    {
        Ds1302ReadTime();
        DisplayData[0] = DIG_CODE[TIME[2]/16];              //时
        DisplayData[1] = DIG_CODE[TIME[2]&0x0f];
        DisplayData[2] = 0x40;
        DisplayData[3] = DIG_CODE[TIME[1]/16];              //分
        DisplayData[4] = DIG_CODE[TIME[1]&0x0f];
        DisplayData[5] = 0x40;
        DisplayData[6] = DIG_CODE[TIME[0]/16];              //秒
        DisplayData[7] = DIG_CODE[TIME[0]&0x0f];
        DigDisplay();
    }

}
```

C51 的数据类型

- **常量—在程序运行中其值不变的量，可以为字符，十进制数或十六进制数 (用 0x 表示)**

常量分为数值型常量和符号型常量，如果是符号型常量，需用宏定义指令 (#define) 对其进行定义 (相当于汇编的 “ EQU” 伪指令) 如：

```
#define  PI  3.1415
```

那么程序中只要出现 PI 的地方，编译程序都译为 3.1415 。

- **变量—在程序运行中其值可以改变的量**

一个变量由变量名和变量值构成，变量名即是存贮单元地址的符号表示，而变量的值就是该单元存放的内容。定义一个变量，编译系统就会自动为它安排一个存贮单元，具体的地址值用户不必在意。

C51 的数据类型

	数 据 类 型	长 度	值 域
位 型	bit	1Bit	0 或 1
字符型	signed char	1Byte	-128~127
	unsigned char	1Byte	0~255
整型	signed int	2Byte	-32768~+32767
	unsigned int	2Byte	0~65535
	signed long	4Byte	-2147483648~+2147483647
	unsigned long	4Byte	0~4294967295
实型	Float	4Byte	1.176E-38~3.40E+38
指针型	data/idata/ pdata	1Byte	1 字节地址
	code/xdata	2Byte	2 字节地址
	通用指针	3Byte	其中 1 字节为储存器类型编码，2,3 字节为地址偏移量
访问 SFR 的数据类型	sbit	1Bit	0 或 1
	sfr	1Byte	0~255
	sfr16	2Byte	0~65535

C51 编程要求

- 字符型 (char) 、整型 (int) 和长整型 (long) 均有符号型 (signed) 和无符号型 (unsigned) 两种，如果不是必须，尽可能选择 unsigned 型，这将会使编译器省却符号位的检测，使生成的程序代码比 signed 类型短得多
- 程序编译时， C51 编译器会自动进行类型转换，例如将一个位变量赋值给一个整型变量时，位型值自动转换为整型值；当运算符两边为不同类型的数据时，编译器先将低级的数据类型转换为较高级的数据类型，运算后，运算结果为高级数据类型。
- 51 单片机内部数据存贮器的可寻址位 (20H ～ 2FH) 定义为 bit 型，而特殊功能寄存器的可寻址位 (即地址为 X0H 和 X8H 的 SFR 的各位) 只能定义为 sbit 类型

C51 编程中的指针

- 在汇编语言程序中，要取存贮单元 m 的内容可用直接寻址方式，也可用寄存器间接寻址方式 ，如果用 R1 寄存器指示 m 的地址，用 @R1 取 m 单元的内容
- C 语言中用变量名表示取变量的值 (相当于直接寻址) ，也可用另一个变量 (如 P) 存放 m 的地址， P 就相当于 R1 寄存器 。用 *P 取得 m 单元的内容 (相当于汇编的间接寻址方式) 这里 P 即为指针型变量

直接寻址		间 接 寻 址	
汇编语言	C语言	汇编语言	C语言
mov n,m 传送语句	n=m; 赋值语句	mov R1,#m ; m的地址送R1 mov n,@R1 ; m的内容送n	P=&m /*m的地址送P*/ n=*P /*m的内容送n*/

C51 编程中的指针

- 51 单片机的不同存贮空间，有不同的地址范围
- 指针类型的定义中要说明：被指的变量的数据类型和存贮类型；指针变量本身的数据类型（占几个字节）和存贮类型（即指针本身存放在什么存贮区）
- 类型定义为 data 或 idata，表示指针指示内部数据存贮器；pdata 表示指针指向外部数据存贮器，以上均为八位地址；而类型 code/xdata 表示指针指向外部程序存贮器或外部数据存贮器指针，本身（即被指 示地址）应为十六位长度。

存贮器类型	idata	xdata	pdata	data	code
编　码	1	2	3	4	5

C51 的变量的存贮类型

存贮器类型	描　　述
data	直接寻址内部数据存贮区，访问变量速度最快(128bytes)
bdata	可位寻址内部数据存贮区，允许位与字节混合访问(16bytes)
idata	间接寻址内部数据存贮区，可访问全部内部地址空间(256bytes)
pdata	分页(256bytes)外部数据存贮区，由操作码MOVX@Ri访问
xdata	外部数据存贮区(64K)，由操作码MOVX@DPTR访问
code	代码存贮区(64K)，由操作码MOVC@A+DPTR访问

C51 的运算符

- **赋值运算符：＝**

  将“＝”的右边的值赋值给左边的变量 .

- **C51 的算术运算符：**

＋（加或正号）；－（减或负号）；＊（乘号）；／（除号）；％（求余）

优先级为：先乘除，后加减，先括号内，再括号外

- **C51 的关系运算符有六种：**

＜（小于）；＞（大于）；＜＝（小于等于）；＞＝（大于等于）; ==（相等）；!=（不相等）

优先级：前四个高，后二个“==”和“!=”级别低。

C51 的运算符

- **C51 的逻辑运算符有三种：**

  &&(逻辑与)； ||(逻辑或)； !(逻辑非)

- **C51 的按位操作的运算符有六种：**

&(按位与)； | (按位或)； ^ (按位异或)； ～ (位取反)；

<<(位左移)； >>(位右移) (注：补零移位)

例 1. a=0xf0H ；表达式 a= ～ a 值为 0FH

例 2. a=0xea ；表达式 a<<2 值为 A8H ，即 a 值左移两位，移位后空白位补 0

- **自增、自减运算符：**

++i ， --i (在使用 i 之前，先使 i 值加 1, 减 1)

i++ ， i-- (在使用 i 之后，再使 i 值加 1., 减 1)

例设 i 原值为 5 j=++i 则 j 值为 6 ， i 值也为 6

j=i++ 则 j 值为 5 ， i 值为 6

C51 的运算符

□ 复合赋值运算符：

+=；－=；*=；/=；%=；<<=；>>=；
&=；^=；|=。

例： a+=b 相当于 a=a+b 。

a>>=7. 相当于 a=a >>7. 。

□ 对指针操作的运算符：

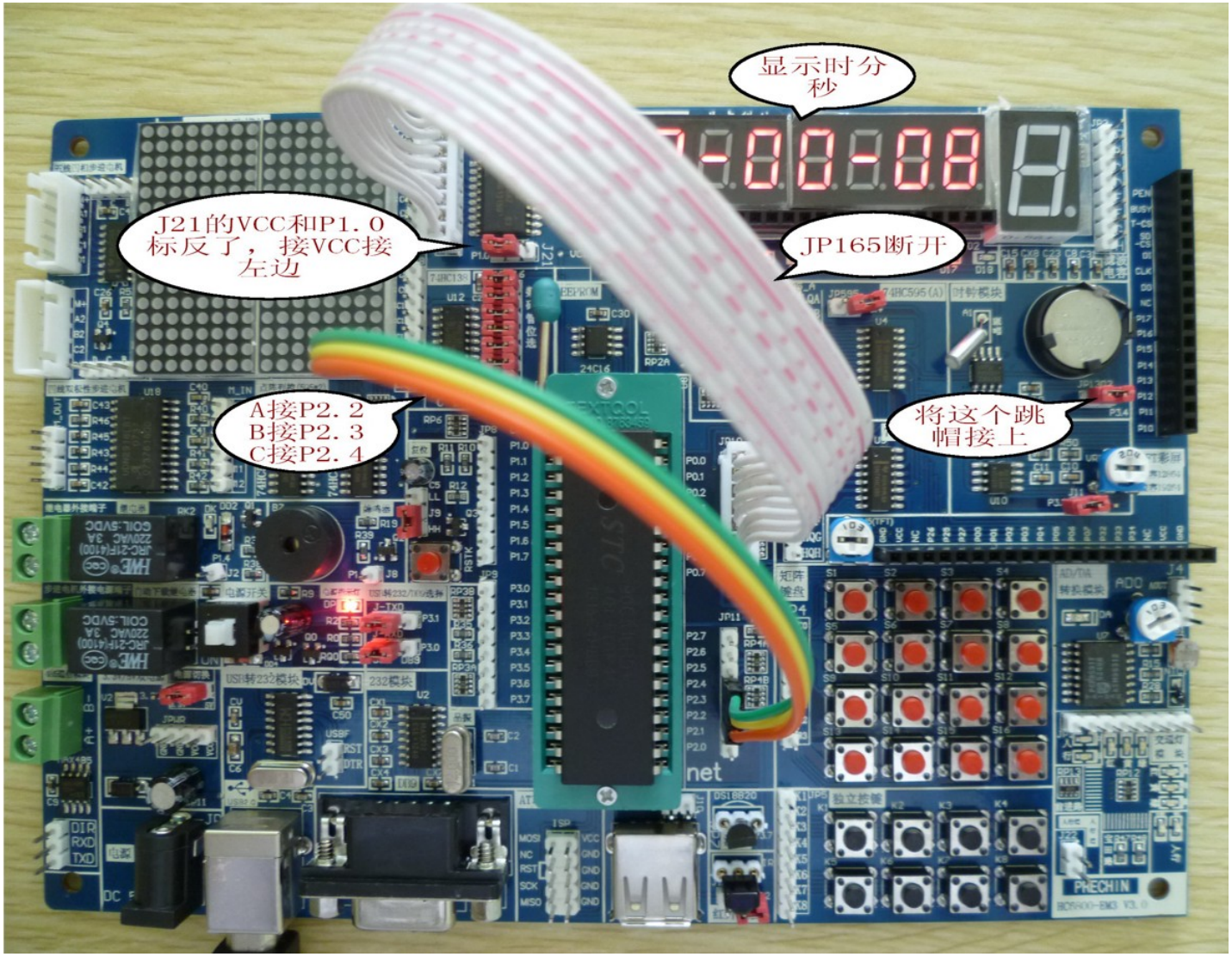
&— 取地址运算

*— 间址运算符

例 a=&b；取 b 变量的地址送变量 a

c=*b；将以 b 的内容为地址的单元的内容送 c

DS1302 读写及时间显示试验



试验要求

- 在实验板上编辑编译 C 语言程序，下载运行，检查是否得到预期的结果
- 对照 PPT 、 DS1302 的数据手册（中文、英文），充分理解代码原理
- 对每条语句进行详细注释，以加深对程序原理的理解
- 修改为当前时间，检查是否达到预期
- 用示波器同时采集“ I/O”和“ SCLK”信号，对照示波器，讲解单片机读 / 写 DS1302 的时序过程
- 测试提供的汇编代码，修正可能存在的问题，对每条语句进行详细注释。

流程图示例

